Quantum Neural Networks in the NISQ Era: Architectures, Challenges, and Applications

Kanishka W. Palihakkara[1] and Mahesh N. Jayakody[2]

[1] Department of Marine Engineering, CINEC Campus, Sri Lanka
[2] Forschungszentrum Jülich, Wilhelm-Johnen-Straße, Jülich, Germany
palihakkara351@gmail.com

Abstract. Quantum Neural Networks (QNNs) have emerged as a promising paradigm at the intersection of quantum computing and machine learning. As we advance through the Noisy Intermediate-Scale Quantum (NISQ) era, these models offer both opportunities and challenges. On the one hand, QNNs promise novel ways of representing data and solving problems in domains ranging from pattern recognition to scientific simulation. On the other, their development is hindered by practical constraints such as hardware noise, limited qubit counts, and difficulties in training. This paper provides a concise overview of QNN architectures and training strategies, highlights key open challenges, and discusses their implications for near-term quantum computing.To make the field more accessible, we also provide a tutorial-style example that constructs and trains a simple four-qubit QNN classifier on the two-moons dataset. In this illustrative example, the QNN model achieves a test accuracy in the range of $92 \pm 2\%$, and we include a small multilayer perceptron as a classical baseline to contextualize performance. Our goal is to present a balanced resource that both introduces the fundamentals and reflects the current perspectives serving as a guide for newcomers and practitioners.

Keywords: Quantum Neural Networks · Variational Quantum Circuits · Quantum Machine Learning · Noisy Intermediate-Scale Quantum computing

1 Introduction

Quantum computing has rapidly evolved from a theoretical framework into an emerging computational paradigm that promises capabilities beyond those of classical systems. Among its most promising applications is quantum machine learning (QML), which seeks to exploit quantum resources to enhance learning algorithms [1, 2]. Within this context, Quantum Neural Networks (QNNs) have attracted significant attention as hybrid architectures that combine the representational power of neural networks with the unique properties of quantum mechanics [3, 4].

QNNs can be understood as a subclass within the broader family of Variational Quantum Algorithms (VQAs), since most practical implementations adopt the variational paradigm of parameterized quantum circuits trained with classical optimizers. VQAs are hybrid quantum–classical methods in which the parameters of a quantum circuit are optimized by a classical routine to extremize an objective function derived from measurement outcomes [5, 6]. Prominent examples include the Variational Quantum Eigensolver (VQE), designed for ground-state energy estimation [7], and the Quantum Approximate Optimization Algorithm (QAOA), which addresses combinatorial optimization problems. In these cases, the objective is typically physics- or problem-driven (e.g., minimizing an expected energy or a cost Hamiltonian) and does not necessarily involve data or labels. By contrast, QNNs specialize the VQA framework for learning from data, in which classical inputs are encoded into quantum states, a variational ansatz transforms these states into expressive features, and measurement statistics guide the optimization of model parameters with respect to learning objectives such as supervised losses or unsupervised divergences. Thus, while many QNNs take the form of VQAs, the broader VQA family also encompasses physics- and optimization-oriented algorithms that do not qualify as QNNs. The present hardware landscape, known as the Noisy Intermediate-Scale Quantum (NISQ) era, offers devices with tens to hundreds of qubits that are noisy and not fault-tolerant [8]. These constraints have motivated hybrid approaches such as VQCs, where parameterized unitaries are trained by classical optimizers using measurements from quantum devices or simulators [5, 6]. QNNs fit naturally into this paradigm as data-driven variational models.

A growing body of research has examined the representational power, training dynamics, and applications of QNNs. Havlíček et al. [9] introduced quantum-enhanced feature maps for supervised learning, while Benedetti et al. [10] analyzed the expressibility of parameterized quantum circuits as machine learning models. Abbas et al. [11] compared QNNs with classical counterparts and demonstrated potential advantages in representational capacity. However, McClean et al. [12] identified a major obstacle in the form of barren plateaus, where gradients vanish exponentially with system size, limiting trainability. On the applications side, Kandala et al. [13] applied variational methods to simulate molecular systems, and Du et al. [14] proposed quantum generative models for small-molecule drug discovery. In parallel, Schuld and Killoran [15] formalized the use of Hilbert-space embeddings as the basis for quantum kernel methods, connecting QNNs with broader machine learning frameworks. QNNs are particularly attractive for two main reasons. First, they enable quantum-enhanced representations via feature maps and entangling operations that can realize hypothesis classes difficult to emulate classically [9–11,16]. Second, their hybrid nature aligns with near-term hardware by keeping circuits relatively shallow and delegating heavy optimization to classical processors [6]. While most QNN architectures can be understood within the VQA framework, other quantum learning approaches such as kernel-based methods [9,15] and reservoir computing [17,18] do not strictly fall into this category. Nevertheless, within the variational paradigm that underlies the majority of QNNs, a diverse range of architectural proposals has been developed, spanning hardware-efficient ansätze and data re-uploading schemes to hierarchical designs such as Quantum Convolutional Neural Networks (QCNNs) [4,15,19]. These models have been explored for classification and pattern recognition, quantum chemistry and materials, and generative modeling, among other areas [13,14,20]. Despite their promise, QNNs face significant challenges. One major obstacle is the occurrence of barren plateaus, a phenomenon in which the gradients of the objective function vanish exponentially with system size or circuit depth, severely limiting scalability [6, 12, 19]. In addition, hardware noise, restricted qubit connectivity, and shot noise further hinder both optimization and generalization [13, 21]. Moreover, benchmarking against strong classical baselines remains an open issue; several studies caution that reported advantages may depend sensitively on data encodings, circuit choices, or evaluation protocols [14, 16, 20].

This paper presents a concise, practice-oriented overview of QNNs in the NISQ era. We review core architectures, data-encoding schemes, and variational ansätze, highlighting the trainability–expressivity trade-offs imposed by current hardware. Moreover, we survey training methodologies and noise-aware practices suited to realistic experiments. To lower the entry barrier, we provide two tutorial-style examples that demonstrate an end-to-end QNN workflow using open-source tools—from encoding and circuit design to training and evaluation. Overall, our aim is to offer a balanced resource that covers fundamentals, clarifies practical constraints, and outlines opportunities for QNN research and applications in NISQ era.

2 QNN Architectures

The design of QNNs is closely tied to the framework of variational quantum circuits, where parameterized gates serve as tunable elements analogous to weights in classical networks. These VQA architectures typically follow a layered structure, in which quantum gates are applied to qubits, interleaved with entangling operations to capture correlations across the system. A classical optimizer updates the parameters based on objective values computed from quantum measurements, thereby completing the learning loop [5, 6]. A schematic diagram of hybrid quantum–classical learning loop is given in Fig 1. This flexible structure has led to a variety of QNN models with distinct representational properties and application domains. Another promising class of architectures of QNNs is the QCNNs, proposed as a hierarchical model that mirrors the convolutional and pooling layers of classical deep learning. QCNNs exploit translational symmetry and entangling gates to reduce dimensionality while preserving relevant features, offering efficiency advantages in data processing tasks [4]. Simulation studies and early hardware demonstrations suggest that QCNNs may be well suited for near-term devices, particularly in pattern recognition and quantum phase classification problems. Hybrid quantum–classical models appears as another important architectural approach in which quantum layers are embedded within larger classical neural networks, allowing quantum circuits to focus on specific subroutines such as feature extraction or kernel evaluation [15,22]. This design utilizes the strengths of both paradigms, enabling the integration of quantum resources without requiring full-scale quantum systems. Such hybrid architectures are particularly attractive in the NISQ setting, where quantum resources are limited and must be used sparingly.

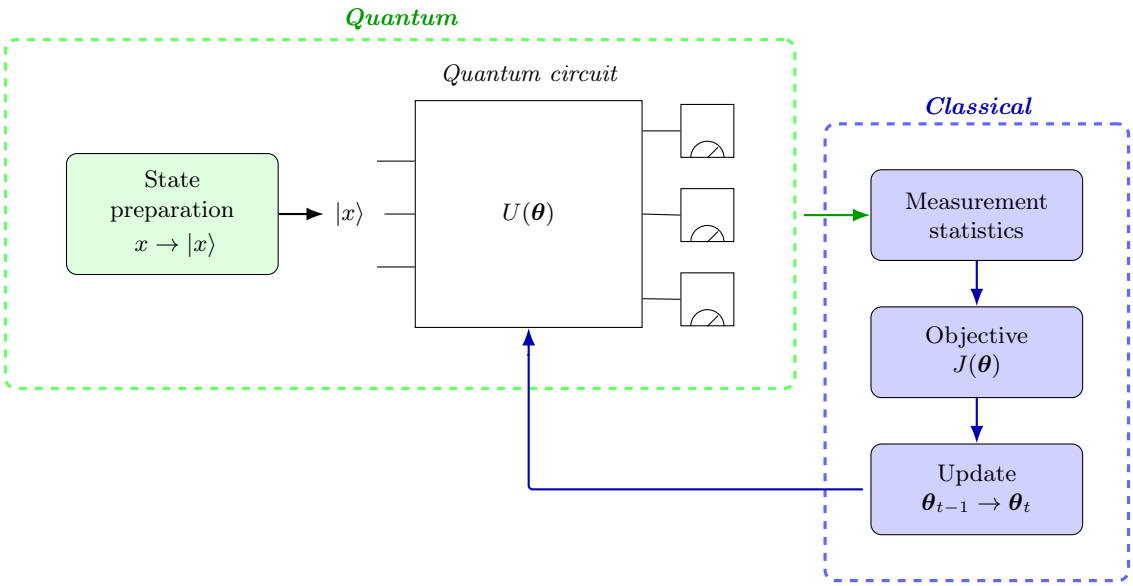

Fig. 1. Schematic diagram of quantum–classical learning loop. Classical inputs are encoded as quantum states ($x \to |x\rangle$), processed by a parameterized circuit $U(\boldsymbol{\theta})$, and measured. The resulting statistics feed the classical side, which computes an objective $J(\boldsymbol{\theta})$ and updates the parameters $\boldsymbol{\theta}_{t-1} \to \boldsymbol{\theta}_t$, closing the loop. In most settings, the state-preparation map is fixed (non-trainable), and only the quantum circuit is optimized. In extended models, the state-preparation map may also include trainable parameters, where the optimization feedback applies to both encoding and quantum circuit layers. Optimization targets depend on the task: supervised learning minimizes a loss (e.g., mean squared error or cross-entropy); unsupervised/generative training uses a likelihood, divergence, or reconstruction error and reinforcement learning maximizes expected return.

Beyond these core models, alternative paradigms inspired by classical machine learning are also being investigated. Examples include quantum recurrent neural networks for sequential data [23], graph-based quantum neural networks inspired by graph convolutional methods [24], and tensor-network-inspired designs that link QNNs with condensed matter approaches [25]. In parallel, quantum reservoir computing has been proposed as a framework in which a fixed, high-dimensional quantum system acts as the reservoir and only the classical readout layer is trained [17,18]. While not variational in nature, reservoir-based approaches offer an alternative route to exploiting quantum dynamics for learning. Collectively, these developments illustrate the expanding landscape of QNN-inspired models and the creative interplay between quantum information theory and machine learning. Throughout the manuscript, we interpret QNNs as data-driven variational quantum circuits, and we explicitly distinguish these models from related quantum learning approaches such as kernel methods or reservoir computing, which do not rely on variational optimization.

3 Mathematical Background of QNNs

Among different architectures, the most widely used approaches to QNNs rely on the VQA framework. As illustrated in Fig. 1, these models comprise three main stages: (i) classical data are encoded into quantum states via a state-preparation map (encoder), (ii) a parameterized quantum circuit (ansatz) transforms these states to generate expressive quantum features, and (iii) measurement outcomes are processed by a classical optimizer, which evaluates an objective function and updates the parameters, closing the hybrid learning loop. Below we provide a mathematical formulation of this workflow.

State preparation : Let $\mathbf{x} \in \mathbb{R}^n$ denote a classical input vector. The first stage encodes $\mathbf{x}$ into a quantum state using a feature map $S(\mathbf{x})$, producing

$$|\phi(\mathbf{x})\rangle \;=\; S(\mathbf{x})\,|0\rangle^{\otimes q} \tag{1}$$

on q qubits [9, 15]. In most VQA settings, $S(\mathbf{x})$ is fixed and non-trainable, though in some variants the encoder itself may contain learnable parameters.

Variational ansatz : The second stage applies a parameterized unitary $U(\boldsymbol{\theta})$ to the encoded state,

$$|\psi(\mathbf{x}, \boldsymbol{\theta})\rangle \;=\; U(\boldsymbol{\theta})\,|\phi(\mathbf{x})\rangle \tag{2}$$

where $\boldsymbol{\theta}$ denotes the set of trainable parameters.

Measurement and forward map : Model predictions are obtained by measuring observables $M = \sum_j \alpha_j P_j$, where P_j are Pauli strings. The expectation value defines the QNN forward map as

$$f(\mathbf{x}, \boldsymbol{\theta}) = \langle\psi(\mathbf{x}, \boldsymbol{\theta})|\, M\, |\psi(\mathbf{x}, \boldsymbol{\theta})\rangle\,. \tag{3}$$

This corresponds to the "Measurement statistics" block in Fig. 1, with the exact form of M chosen according to the learning task at hand. This formulation, state preparation (feature map), variational ansatz, and measurement provides a unified template that encompasses all QNN architectures considered in this work.

Objective functions: We denote the trainable objective by $J(\boldsymbol{\theta})$, following the notation in Fig. 1. In the context of machine learning, this objective typically corresponds to a *loss function*, whose form depends on the task. In supervised learning, $J(\boldsymbol{\theta})$ is the empirical risk, such as cross-entropy for classification or mean squared error for regression. In unsupervised or generative learning, it may represent a negative log-likelihood, a divergence (for example, KL divergence), or a reconstruction error. In reinforcement learning, $J(\boldsymbol{\theta})$ is commonly defined as the negative expected return, so that minimizing J corresponds to maximizing cumulative reward.

Gradients and parameter update : Training requires gradients of (3) with respect to $\boldsymbol{\theta}$. In the context of variational circuits, a method called the *parameter-shift rule* can be employed to evaluate these gradients directly on quantum hardware. If we are interested in the derivative with respect to the k-th gate's angle, the circuit can be factored as $U(\boldsymbol{\theta}) = VG_k(\theta_k)\,W$ where W denotes all gates before the k-th gate (including state preparation) and V denotes all gates after the k-th gate (including operations before measurement). When taking the partial derivative $\partial/\partial\theta_k$, the operators V and W are treated as fixed (they may depend on other parameters, but those are held constant for this derivative) [26]. A classical optimizer such as Adam or L-BFGS is used to updates the parameters as $\boldsymbol{\theta}^* \;=\; \arg\min_{\boldsymbol{\theta}}\; J(\boldsymbol{\theta})$. This optimization problem defines the learning process of a QNN. The alternating quantum–classical loop of state preparation, circuit evaluation, measurement, and parameter update continues until convergence criteria are met or a predefined number of iterations is reached.

4 Data Encoding in QNNs

Data encoding in QNNs refers to the process of mapping classical data into quantum states so that a QNN can exploit superposition, entanglement, and interference for learning. The choice of encoding strongly impacts circuit width (qubits), depth (gates), noise sensitivity, trainability, and ultimately the efficiency and scalability of a model on NISQ hardware [9, 15, 22]. In practice, encoding is often combined with classical preprocessing, such as normalization or dimensionality reduction, to ensure numerical stability and hardware feasibility. Formally, a feature vector $\mathbf{x} \in \mathbb{R}^n$ is embedded into a quantum state through a state-preparation operator $S(\mathbf{x})$, yielding $|\phi(\mathbf{x})\rangle = S(\mathbf{x})\,|0\rangle^{\otimes q}$. The operator $S(\mathbf{x})$ may be fixed or trainable, but in either case its construction often dominates the overall resource requirements of the model.

Several encoding families have been explored in the literature. *Angle encoding* maps features to single-qubit rotations, with variants including multi-angle encoding, repeated (layered) encoding, and hybrid combinations. A commonly employed single-qubit angle encoding scheme is given by

$$|\phi(\mathbf{x})\rangle = \bigotimes_{i=0}^{n-1} R_y(x_i)\,|0\rangle^{\otimes n} \tag{4}$$

where $\mathbf{x} = (x_0, x_1, \ldots, x_{n-1})$ is the classical feature vector with n components, and each feature x_i is injected as a rotation angle on a separate qubit and R_y is the rotation gate around the Y-axis of the Bloch sphere. This construction yields shallow, hardware-efficient circuits but requires one qubit per feature (or per re-uploaded block). A richer variant, often called multi-angle encoding, applies more than one rotation per feature as

$$|\phi(\mathbf{x})\rangle = \bigotimes_{i=0}^{n-1} R_y(x_i) R_z(x_i) \, |0\rangle^{\otimes n} \tag{5}$$

where both R_y and R_z rotations are used to encode each component. Such schemes can increase expressivity without increasing the qubit count, though at the cost of slightly deeper circuits. *Amplitude encoding* represents a classical vector $\mathbf{x} = (x_0, x_1, \ldots, x_{n-1}) \in \mathbb{R}^n$ as the amplitudes of a quantum state. Without loss of generality, we assume $n = 2^m$ (padding with zeros if necessary), which allows the encoding

$$|\phi(\mathbf{x})\rangle = \frac{1}{\|\mathbf{x}\|_2} \sum_{j \in \{0,1\}^m} x_{\mu(j)} \, |j_0 j_1 \cdots j_{m-1}\rangle \tag{6}$$

where $j = (j_0, j_1, \ldots, j_{m-1})$ with each $j_k \in \{0, 1\}$ and $\mu(j) = \sum_{k=0}^{m-1} j_k 2^{m-1-k}$ and the normalization factor is given by $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=0}^{n-1} |x_i|^2}$. In this way, an n-dimensional input vector $\mathbf{x}$ can be embedded into $m = \log_2 n$ qubits. It is exponentially compact in qubit count but often requires deep, noise-sensitive state preparation. *Basis encoding* directly maps discrete or binary features to computational basis states. Suppose we have a binary feature vector $\mathbf{x} = (x_0, x_1, \ldots, x_{n-1})$ with $x_i \in \{0, 1\}$. In basis encoding, $\mathbf{x}$ is mapped directly to the computational basis state as

$$|\phi(\mathbf{x})\rangle = |x_0 x_1 \cdots x_{n-1}\rangle \tag{7}$$

where each entry x_i specifies the state of qubit i. This approach is conceptually simple and well suited for discrete or combinatorial inputs, but it requires one qubit per encoded bit and has limited expressivity without additional variational layers.

5 Variational Ansatz

A variational ansatz specifies the parameterized unitary $U(\boldsymbol{\theta})$ that acts on encoded input states to generate task-relevant features. Its design determines the expressive power of the model, the required circuit depth, and the degree of entanglement, while also being constrained by hardware connectivity and noise on NISQ devices. In practice, variational ansatz are typically constructed as layered circuits consisting of alternating single-qubit rotations and entangling operations. One of the simplest and most widely adopted choices is the *Basic Entangler Layers* ansatz, where each qubit undergoes a parameterized R_y rotation followed by a linear chain of CNOT gates (Fig. 2).

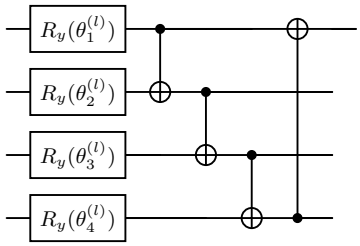

Fig. 2. Illustration of the ℓth layer in the *Basic Entangler Layer* ansatz with four qubits. Each qubit undergoes a single-qubit R_y rotation, followed by a nearest-neighbor chain of CNOT gates that entangles the register in a linear topology. The final wrap-around CNOT connection between the last and the first qubit ensures global entanglement.

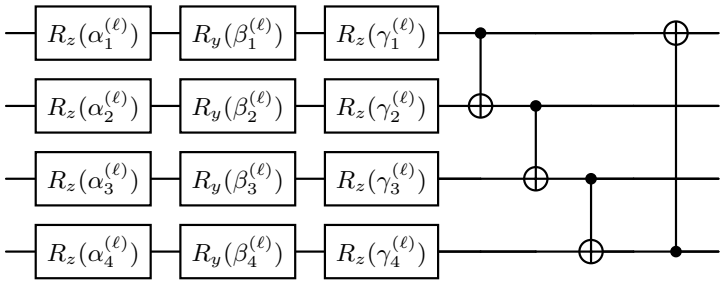

Fig. 3. The ℓth layer of a *Strongly Entangling Layer* ansatz on four qubits. Each qubit undergoes a full set of single-qubit Euler rotations (R_z–R_y–R_z), followed by a brickwork of CNOT gates that includes a wrap-around connection between the last and the first qubit, ensuring dense entanglement across the register [27].

While easy to implement and hardware-friendly, this design may lack expressivity for complex tasks. To address this, the *Strongly Entangling Layers* ansatz introduces full Euler decompositions (R_z–R_y–R_z) on each qubit together with a brickwork of CNOT gates, including wrap-around connections, thereby enhancing expressivity at the cost of increased circuit depth (Fig. 3). A related direction is provided by *Hardware-Efficient Ansätze*, such as Qiskit's `EfficientSU2`, which adopt parameterized single-qubit rotations interleaved with entanglers that follow the hardware connectivity graph. These designs strike a balance between expressivity and near-term implementability. More structured architectures such as QCNNs, recurrent quantum circuits, or problem-inspired ansatz like QAOA can be viewed as extensions or specializations of these baseline forms. The choice of ansatz should be guided by hardware constraints and the learning task at hand.

6 Training Strategies for QNNs

The effective training of QNNs is one of the central challenges in the NISQ era, as optimization procedures must contend with both the limitations of current hardware and the complexity of quantum parameter landscapes. In most cases, QNNs adopt a hybrid training loop, where a parameterized quantum circuit evaluates the objective function and a classical optimizer updates the parameters based on gradient information or alternative heuristics [5,6]. Gradient-based methods such as stochastic gradient descent and Adam are among the most widely used approaches. In QNNs, these methods are enabled by techniques like the parameter-shift rule, which provides an exact way to compute gradients in variational circuits by evaluating shifted versions of the circuit [26]. In its simplest form, parameter-shift rule can be stated as follows. Let $G(\alpha) = e^{-i\alpha H/2}$ be a single-parameter gate with Hermitian generator H. Suppose the generator H has only two distinct eigenvalues $\pm\lambda$. Then for an observable O and state $|\phi\rangle$, the expectation value $g(\alpha) = \langle\phi|\, G_k(\alpha)^\dagger(\alpha)\, O\, G_k(\theta_k)\, |\phi\rangle$ can always be expressed as

$$g(\alpha) = A\cos(\lambda\alpha) + B\sin(\lambda\alpha) + C \tag{8}$$

with real constants λ, A, B, C. This sinusoidal structure leads to the *parameter-shift identity* given by

$$\frac{dg(\alpha)}{d\alpha} = \frac{\lambda}{2}\Big[g\big(\alpha + \tfrac{\pi}{2\lambda}\big) - g\big(\alpha - \tfrac{\pi}{2\lambda}\big)\Big] \tag{9}$$

Hence, the parameter-shift rule provides an exact method for computing gradients using only shifted evaluations of the function, without resorting to numerical differentiation.

A major concern of gradient-based methods is the barren plateau phenomenon, where gradients vanish exponentially with system size, making training intractable for deep circuits [12,28]. When gradients become impractical, gradient-free optimizers such as Nelder–Mead, Powell's method, or evolutionary algorithms can be employed [29,30]. These are more robust to noise but generally scale poorly with dimensionality. Hybrid strategies that combine classical pre-training with quantum fine-tuning have also been suggested [31], as well as noise-aware techniques such as zero-noise extrapolation or probabilistic error cancellation [21,32]. Overall, the landscape of QNN training strategies reflects a trade-off between efficiency, robustness, and scalability. While gradient-based methods remain dominant, the combination of error mitigation, ansatz

design, and hybrid optimization techniques continues to shape the development of practical QNN training pipelines. As quantum hardware improves, the interplay between algorithmic advances and noise-resilient training strategies will be critical to unlocking the potential of QNNs in real-world applications.

7 Challenges in the NISQ Era

Despite their promise, QNNs face significant obstacles in the NISQ era, where devices are limited in qubit number, coherence time, and connectivity. Noise and decoherence represent perhaps the most immediate challenge, as quantum states rapidly degrade under environmental interaction and imperfect gate operations. Even shallow variational circuits accumulate errors that distort training outcomes, necessitating error mitigation techniques and noise-aware algorithm design [8,21,32]. Another central issue is scalability. Current hardware typically supports only tens to low hundreds of qubits, with restricted connectivity that constrains circuit layout. Mapping QNN architectures onto such devices often requires additional swap operations, which further increase circuit depth and noise accumulation [13]. This makes it difficult to realize expressive models without overwhelming hardware limitations. Furthermore, as system size increases, optimization becomes more challenging due to the barren plateau phenomenon, where gradients vanish exponentially with depth or qubit count, impeding effective training [12,28]. Benchmarking and evaluation add another layer of difficulty. Unlike classical machine learning, where well-established datasets and performance metrics exist, quantum machine learning lacks standardized benchmarks that can reliably demonstrate quantum advantage. Several studies have suggested that many near-term QNN experiments may not outperform classical counterparts when evaluated fairly [33–35]. This ambiguity complicates claims of practical utility and highlights the importance of careful comparisons to classical baselines. The complexity of optimization landscapes in QNNs also introduces challenges that are both algorithmic and physical. Training often requires a large number of circuit evaluations, which can be prohibitively expensive on quantum hardware. Classical optimizers may struggle with noisy cost functions and non-convex landscapes, leading to slow or unstable convergence [36]. Addressing these challenges will require innovations in ansatz design, error mitigation, and optimization theory, as well as continued progress in quantum hardware engineering. Together, these limitations define the boundaries of what is currently feasible for QNNs, while also motivating new strategies for overcoming them in the path toward practical quantum machine learning.

8 Applications of QNNs

QNNs have been explored across a wide range of applications, with the dual goals of demonstrating near-term utility and probing the potential for long-term quantum advantage. One of the most prominent areas is classification and pattern recognition, where QNNs have been used for tasks such as image recognition, medical diagnosis, and signal processing [3, 11, 37]. In these settings, QNNs operate either on classical data encoded into quantum states or directly on quantum data, providing an opportunity to investigate whether quantum-enhanced feature spaces can offer improvements over conventional machine learning models. Another important application domain is quantum chemistry and materials science. Variational quantum eigensolvers and QNN-inspired architectures have been employed to approximate ground-state energies of molecules and simulate many-body systems [7, 13, 38]. These studies highlight the ability of QNN frameworks to model quantum systems more naturally than classical algorithms, potentially reducing computational overhead in simulating large-scale molecular interactions. Similarly, QNN-based methods have been tested for quantum phase recognition and condensed matter applications, where hierarchical architectures such as Quantum Convolutional Neural Networks (QCNNs) have shown promise [4,39]. Optimization and generative modeling represent additional frontiers for QNN applications. Variational quantum circuits have been employed to tackle combinatorial optimization problems, with results indicating that hybrid approaches can sometimes compete with or surpass classical heuristics in small instances [40]. Generative models based on QNNs have also been proposed for producing quantum states or approximating probability distributions, with potential uses in drug discovery, financial modeling, and synthetic data generation [41,42]. These applications showcase the versatility of QNNs, even if large-scale demonstrations remain constrained by current hardware. QNNs are increasingly being integrated into hybrid quantum–classical pipelines, where quantum components act

as specialized modules within larger classical workflows. This hybridization has been applied in natural language processing, kernel-based learning, and transfer learning scenarios, where QNN layers provide enhanced representational capabilities while relying on classical networks for scalability [15, 31, 43]. Collectively, these efforts illustrate the breadth of domains where QNNs are being investigated, while also underscoring that clear evidence of quantum advantage remains limited. Nonetheless, the diversity of applications demonstrates the adaptability of QNNs and motivates further exploration as quantum hardware continues to improve.

9 Tutorial-style Examples

In this section, we present two tutorial-style examples aimed at improving the comprehensibility of QNNs for newcomers and practitioners alike. The first example demonstrates how to evaluate the expectation value of a chosen observable using a variational circuit, and how the parameter-shift rule can be applied to compute gradients. The second example illustrates the complete workflow of constructing and training a variational quantum circuit for a binary classification task. Here, the goal is not to achieve state-of-the-art performance, but rather to provide a clear, hands-on introduction. For this purpose, we employ the `PennyLane` library [44], which enables seamless integration of quantum circuits with classical machine learning platforms.

Example 1:

Let us consider the toy ansatz given in Fig. 4 which defines a minimal two-qubit ansatz with two trainable parameters. On the first qubit, a single-qubit rotation $R_y(\theta_1)$ is applied, which prepares a superposition state depending on the variational parameter θ_1. A controlled-NOT (CNOT) gate then entangles the first qubit with the second qubit, distributing information across the system. After the entangling operation, the second qubit undergoes another rotation $R_y(\theta_2)$, controlled by the second variational parameter.

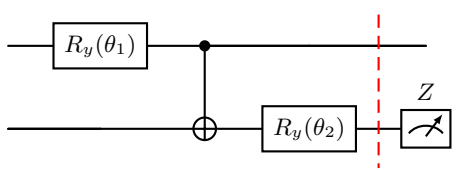

Fig. 4. Two-qubit variational ansatz with two trainable parameters

The variational quantum circuit given in Fig. 4 can be expressed in operator form as

$$U(\theta_1, \theta_2) = \left[\mathbb{I} \otimes R_y(\theta_2)\right] \text{CNOT} \left[R_y(\theta_1) \otimes \mathbb{I}\right] \tag{10}$$

with $\mathbb{I}$ denoting the identity operator and $\otimes$ represents the tensor product. The CNOT gate is defined as

$$\text{CNOT} = |0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes X \tag{11}$$

where X is the Pauli-X (NOT) and the operators $|0\rangle\langle 0|$, $|1\rangle\langle 1|$ are projectors onto the computational basis states. The matrix form of each gate is straightforward. The R_y rotation acting on a single qubit is given by

$$R_y(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}. \tag{12}$$

where $\theta \in [0, 2\pi)$. The identity operator, the Pauli-X operator, and the projectors $|0\rangle\langle 0|$, $|1\rangle\langle 1|$ onto the computational basis states are given by

$$\mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad |0\rangle\langle 0| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad |1\rangle\langle 1| = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \tag{13}$$

By substituting these matrices into (11), (10) and applying the matrix tensor product, the overall unitary $U(\theta_1, \theta_2)$ can be written explicitly as a 4×4 matrix acting on the two-qubit Hilbert space as

$$\mathbf{U}(\theta_1,\theta_2) = \begin{pmatrix} \cos\left(\frac{\theta_1}{2}\right)\cos\left(\frac{\theta_2}{2}\right) & -\cos\left(\frac{\theta_2}{2}\right)\sin\left(\frac{\theta_1}{2}\right) & -\cos\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) & \sin\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) \\ \cos\left(\frac{\theta_2}{2}\right)\sin\left(\frac{\theta_1}{2}\right) & \cos\left(\frac{\theta_1}{2}\right)\cos\left(\frac{\theta_2}{2}\right) & -\sin\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) & -\cos\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) \\ -\sin\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) & \cos\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) & -\cos\left(\frac{\theta_2}{2}\right)\sin\left(\frac{\theta_1}{2}\right) & \cos\left(\frac{\theta_1}{2}\right)\cos\left(\frac{\theta_2}{2}\right) \\ \cos\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) & \sin\left(\frac{\theta_1}{2}\right)\sin\left(\frac{\theta_2}{2}\right) & \cos\left(\frac{\theta_1}{2}\right)\cos\left(\frac{\theta_2}{2}\right) & \cos\left(\frac{\theta_2}{2}\right)\sin\left(\frac{\theta_1}{2}\right) \end{pmatrix} \tag{14}$$

Now consider a scenario where both qubits are initialized in the computational basis state $|00\rangle$, and we are interested in evaluating the expectation value of the observable $O = \mathbb{I} \otimes Z$ where Z is the Pauli-Z operator. This can be written in operator form as follows

$$\langle O \rangle_{\theta_1,\theta_2} = \langle 00| \, U^\dagger(\theta_1,\theta_2) \; O \; U(\theta_1,\theta_2) \, |00\rangle \tag{15}$$

where $\langle O \rangle_{\theta_1,\theta_2}$ is the expected value of the observable, $U^\dagger$ and $\langle 00|$ are the conjugate transpose of U and $|00\rangle$ respectively. The matrix representation of $|00\rangle$ and the Pauli-Z operator are given by

$$|00\rangle \;=\; |0\rangle \otimes |0\rangle = \begin{pmatrix}1\\0\end{pmatrix} \otimes \begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}1\\0\\0\\0\end{pmatrix}, \qquad Z \;=\; \begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix} \tag{16}$$

Hence, by substituting (16) and (14) to (15) one can determine the desired expectation value as

$$\langle O \rangle_{\theta_1,\theta_2} = \cos\theta_1 \cos\theta_2 \tag{17}$$

Once the variational ansatz has been evaluated to obtain the expectation value of a chosen observable, the next step is to define an objective (loss) function based on that expectation. In this example, we use

$$\mathcal{L}(\theta_1,\theta_2) \;=\; \tfrac{1}{2}\Big(1 - \langle O \rangle_{\theta_1,\theta_2}\Big) = \tfrac{1}{2}\Big(1 - \cos\theta_1 \, \cos\theta_2\Big) \tag{18}$$

and our goal is to find parameters θ_1 and θ_2 that minimize this loss. Since $\mathcal{L}$ is smooth in both parameters, the partial derivatives can be computed analytically as

$$\frac{\partial \mathcal{L}}{\partial \theta_1} \;=\; \tfrac{1}{2}\,\sin\theta_1 \, \cos\theta_2, \qquad \frac{\partial \mathcal{L}}{\partial \theta_2} \;=\; \tfrac{1}{2}\,\cos\theta_1 \, \sin\theta_2 \tag{19}$$

The same result can be obtained using the parameter-shift rule in (9). In particular,

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \tfrac{1}{2}\Big[\mathcal{L}\big(\theta_1 + \tfrac{\pi}{2}, \theta_2\big) - \mathcal{L}\big(\theta_1 - \tfrac{\pi}{2}, \theta_2\big)\Big] \;=\; \tfrac{1}{2}\,\sin\theta_1 \, \cos\theta_2 \tag{20}$$

$$\frac{\partial \mathcal{L}}{\partial \theta_2} = \tfrac{1}{2}\Big[\mathcal{L}\big(\theta_1, \theta_2 + \tfrac{\pi}{2}\big) - \mathcal{L}\big(\theta_1, \theta_2 - \tfrac{\pi}{2}\big)\Big] \;=\; \tfrac{1}{2}\,\cos\theta_1 \, \sin\theta_2 \tag{21}$$

This simple ansatz captures the essential ingredients of variational quantum circuits: parameterized single-qubit rotations, entangling gates, and measurements. Despite its small size, it provides a nontrivial setting to illustrate the parameter-shift rule, since the expectation value of the measurement depends sinusoidally on the parameters θ_1 and θ_2, allowing analytic gradients to be obtained by shifted circuit evaluations.

Example 2:

In this example, we illustrate the construction and training of a simple QNN classifier, implemented with the `PennyLane` library, to perform a binary classification task. As a benchmark problem, we use the well-known *two-moons* dataset, a nonlinear toy dataset widely employed to evaluate classification models. Each sample is a 2-dimensional feature vector $\mathbf{x} = (x_1, x_2)$ lying on one of two interleaving semicircles, with binary label $y \in \{0, 1\}$ indicating class membership.

***Dataset and Preprocessing*:** For this tutorial experiment, we generate a two-moons dataset consisting of 600 samples with a noise level of 0.15 and a fixed random seed to ensure reproducibility. Both input features are standardized using `StandardScaler()` to keep rotation angles numerically stable. We apply a 75/25 train–test split with stratified sampling to preserve class balance.

***Quantum Ansatz and Data Encoding*:** We define the toy QNN as a variational circuit consisting of alternating layers of parameterized rotations and entangling gates, forming the quantum analogue of a fully connected layer, as shown in Fig. 5. The first step is to encode the classical input features into quantum states. For this purpose, we employ *angle encoding*, where data values directly parameterize single-qubit rotation gates. Since our QNN circuit uses four input qubits, we adopt a feature-duplication strategy to populate all qubits. Specifically, we set $\tilde{x}_1 = \tilde{x}_3 = x_1$ and $\tilde{x}_2 = \tilde{x}_4 = x_2$, with fixed scaling factors $\alpha_i = 1$. This ensures that every qubit carries data-dependent rotations before the variational layers, enabling subsequent entanglers to distribute correlations across the register. The trainable parameters $\theta_{i,j}^{(\ell)}$ are initialized randomly, and the

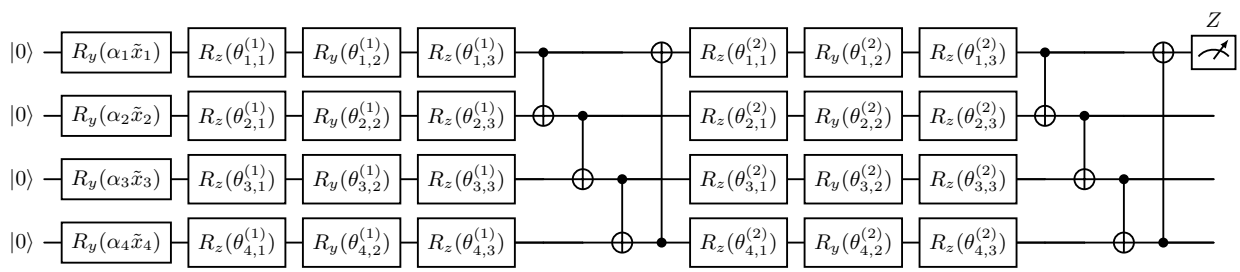

Fig. 5. Model Quantum Neural Network (variational quantum classifier) with four qubits and three layers in total including one encoding layer and two trainable variational layers. The encoding layer performs *angle encoding*, where classical inputs $\tilde{x}_i$ are mapped into quantum states via rotations $R_y(\alpha_i\tilde{x}_i)$. Here α_i are fixed scaling factors controlling how strongly each feature modulates the rotation. Each subsequent block corresponds to a hardware-efficient variational layer: for layer $\ell \in \{1, 2\}$ and qubit i, the trainable parameters $\theta_{i,j}^{(\ell)}$ denote the angles of single-qubit rotations ($j = 1, 2, 3$ for R_z–R_y–R_z). This R_z–R_y–R_z sequence corresponds to the standard Euler decomposition of single-qubit unitaries, ensuring that each variational block can implement any arbitrary single-qubit rotation. Using three parameters per qubit in this way maximizes the expressivity. Entanglement is introduced by a linear chain of CNOT gates across neighboring qubits. In this example, only the first qubit is measured in the Pauli-Z basis, yielding a single scalar expectation value that serves as the model output for binary classification. Measuring a single qubit is common in variational classifiers, as the entangling layers distribute information across all qubits, allowing the network to concentrate predictive power onto one readout; alternatively, one could measure all qubits and combine their expectation values for multi-output or higher-dimensional tasks.

circuit output is obtained through expectation values. We map the Pauli-Z expectation on the first qubit to a probability via

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1 - \langle Z \rangle}{2}, \tag{22}$$

and apply the threshold rule

$$\hat{y}(\mathbf{x}) = \begin{cases} 1, & p_{\boldsymbol{\theta}}(\mathbf{x}) \geq 0.5 \\ 0, & p_{\boldsymbol{\theta}}(\mathbf{x}) < 0.5 \end{cases} \tag{23}$$

which induces the decision boundary $p_{\boldsymbol{\theta}}(\mathbf{x}) = 0.5$.

Loss Function and Training Configuration Training the classifier requires a cost function that quantifies the discrepancy between model predictions and target labels. We adopt the binary cross-entropy loss given by

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log(p_{\boldsymbol{\theta}}(\mathbf{x}_i)) + (1 - y_i) \log(1 - p_{\boldsymbol{\theta}}(\mathbf{x}_i)) \Big] \tag{24}$$

and optimize it using the Adam optimizer (learning rate 0.03), with mini-batches of size 64 over 300 training steps. To assess the stability of training, we repeat the full procedure five times using independent random seeds $\{0, 1, 2, 3, 4\}$ and report the evaluation metric as a mean value along with the standard deviation. It should be noted that, because this example uses a shallow circuit (two variational layers) on only four qubits, it operates in a regime where barren plateaus are not typically observed. This ensures that gradients remain well behaved during training. Moreover, the model is trained using a local cost function (single-qubit Pauli-Z expectation), which is known to mitigate barren plateaus compared to global measurement objectives.

Evaluation Metric Performance is measured using test-set accuracy, defined as

$$\text{Accuracy} = \frac{\text{number of correctly classified test samples}}{\text{total number of test samples}}.$$

Across five independent runs, the QNN achieves a mean test accuracy of 0.92 ± 0.02. Figure 6 illustrates the decision boundary obtained from one representative training run. The model successfully separates the two interleaving classes and generalizes to unseen samples.

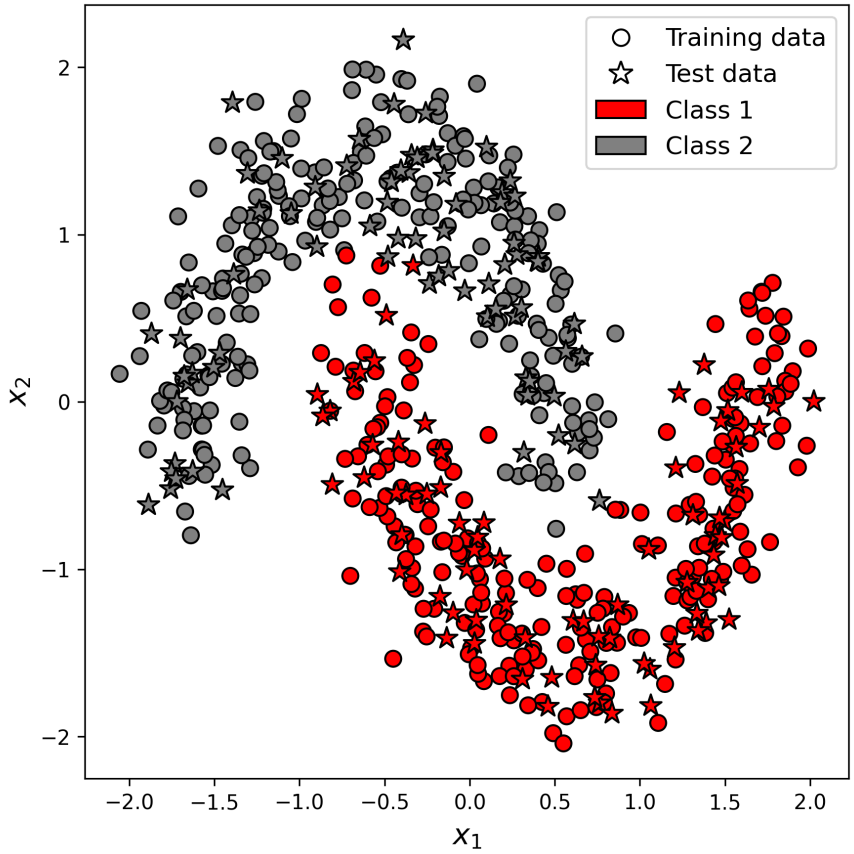

Fig. 6. Decision boundary learned by the QNN on the two-moons dataset (representative run). Training samples are shown as circles and test samples as stars, with red and gray indicating the two classes.

Classical Baseline: To provide a meaningful classical comparison, we implement a small two-layer multi-layer perceptron (MLP) trained on the same standardized two-moons dataset and identical train–test split. The MLP consists of an input layer of dimension 2, followed by two fully connected hidden layers of sizes

16 and 8 with ReLU activations, and a final sigmoid output neuron for binary classification. The model is trained using the Adam optimizer (learning rate 0.01) with a batch size of 64 for 200 epochs. All experiments are repeated using the same set of five random seeds $\{0, 1, 2, 3, 4\}$ to ensure direct comparability with the QNN runs. Under this protocol, the MLP achieves a test accuracy of 0.97 ± 0.01. This baseline highlights that classical models perform strongly on this benchmark, and the QNN should be interpreted primarily as a pedagogical demonstration—rather than as a model intended to exceed classical performance. We emphasize that the tutorial examples presented in this work are intentionally small-scale and rely on noiseless simulation, and therefore should not be interpreted as evidence of quantum advantage or a comparison study with classical MLPs. Instead, they are intended purely as pedagogical demonstrations that reflect the constraints of the NISQ era and illustrate QNN workflows in a controlled setting.

Although simple, this example illustrates key aspects of QNN development: encoding classical data, designing an ansatz, defining cost functions, and integrating quantum and classical computation. Although we have used the PennyLane library for this tutorial, similar workflows can be implemented in Qiskit Machine Learning [45], TensorFlow Quantum [46], or hybrid PyTorch–quantum interfaces. All code used in this example can be made available upon request to support full reproducibility.

10 Discussion and Conclusions

This paper reviewed core QNN architectures and data-encoding strategies, and discussed training methodologies alongside current limitations. In addition, two tutorial-style examples were provided to demonstrate an end-to-end QNN workflow using open-source tools, with the aim of lowering entry barriers for newcomers to the field. While today's implementations of QNNs remain constrained by NISQ-era hardware, steady advances in error mitigation, ansatz engineering, hybridization, and hardware–algorithm co-design—together with transparent benchmarking—will determine whether QNNs evolve from exploratory testbeds into practical tools with transformative impact across science and industry.

By combining a conceptual overview with hands-on demonstrations, this work seeks to serve as both a reference and a practical guide. In the broader landscape of machine learning, QNNs are unlikely to replace classical methods in the near term, but they hold promise as complementary tools for specialized tasks that exploit uniquely quantum resources. In this context, hybrid quantum–classical models where a classical feature extractor is coupled to a small quantum head, offer a particularly practical near-term route because they reduce the required circuit depth while retaining quantum expressivity. Likewise, quantum kernel methods provide a non-variational alternative that avoids many trainability challenges of VQAs, making them well suited for current hardware. Future research should focus on scalable encoding schemes, robust and noise-aware training pipelines, and rigorous benchmarking frameworks to evaluate genuine quantum advantage. With progress along these directions, QNNs may transition from early-stage experimentation to meaningful contributions in scientific discovery and real-world applications.

References

1. Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. An introduction to quantum machine learning. *Contemporary Physics*, 56(2):172–185, 2015.
2. Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.
3. Edward Farhi and Hartmut Neven. Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002*, 2018.
4. Iris Cong, Soonwon Choi, and Mikhail D Lukin. Quantum convolutional neural networks. In *Nature Physics*, volume 15, pages 1273–1278. Nature Publishing Group, 2019.
5. Jarrod R McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics*, 18(2):023023, 2016.
6. M Cerezo, A Arrasmith, R Babbush, SC Benjamin, S Endo, K Fujii, JR McClean, K Mitarai, XL Yuan, L Cincio, and PJ Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021.
7. Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5:4213, 2014.

8. John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.
9. Vojtěch Havlíček, Antonio D Córcoles, Kristan Temme, Aram W Harrow, Abhinav Kandala, Jerry M Chow, and Jay M Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, 2019.
10. Marcello Benedetti, Delfina Garcia-Pintos, Oscar Perdomo, Vicente Leyton-Ortega, Yunseong Nam, and Alejandro Perdomo-Ortiz. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019.
11. Amira Abbas, David Sutter, Christa Zoufal, Aurelien Lucchi, Alessio Figalli, and Stefan Woerner. The power of quantum neural networks. *Nature Computational Science*, 1(6):403–409, 2021.
12. Jarrod R McClean, Sergio Boixo, Vadim N Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812, 2018.
13. Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature*, 549(7671):242–246, 2017.
14. Yuxuan Du, Min-Hsiu Hsieh, and Tongliang Liu. Quantum generative models for small molecule drug discovery. *npj Quantum Information*, 7(1):33, 2021.
15. Maria Schuld and Nathan Killoran. Quantum machine learning in feature hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019.
16. Hsin-Yuan Huang, Richard Kueng, and John Preskill. Information-theoretic bounds on quantum advantage in machine learning. *Nature Communications*, 12(1):2631, 2021.
17. Keisuke Fujii and Kohei Nakajima. Harnessing disordered-ensemble quantum dynamics for machine learning. *Physical Review Applied*, 8(2):024030, 2017.
18. Kohei Nakajima, Keisuke Fujii, Makoto Negoro, Kosuke Mitarai, and Masahiro Kitagawa. Boosting computational power through spatial multiplexing in quantum reservoir computing. *Physical Review Applied*, 11(3):034021, 2019.
19. Sukin Sim, Peter D Johnson, and Alán Aspuru-Guzik. Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Advanced Quantum Technologies*, 2(12):1900070, 2019.
20. Michael Lubasch, Jaewoo Joo, Thibault Moinier, Martin Kiffner, and Dieter Jaksch. Variational quantum algorithms for nonlinear problems. *Physical Review A*, 101(1):010301, 2020.
21. Kristan Temme, Sergey Bravyi, and Jay M Gambetta. Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119(18):180509, 2017.
22. Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R. McClean. Power of data in quantum machine learning. *Nature Communications*, 12(1):2631, 2021.
23. Johannes Bausch. Recurrent quantum neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1368–1379, 2020.
24. Guillaume Verdon, Michael Broughton, and Jacob Biamonte. Quantum graph neural networks. *arXiv preprint arXiv:1909.12264*, 2019.
25. William Huggins, Pranav Patil, Brian Mitchell, K. Birgitta Whaley, and Miles Stoudenmire. Towards quantum machine learning with tensor networks. *Quantum Science and Technology*, 4(2):024001, 2019.
26. Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331, 2019.
27. Jonathan Foldager and Bálint Koczor. Can shallow quantum circuits scramble local noise into global white noise? *Journal of Physics A: Mathematical and Theoretical*, 57(1):015306, 2023.
28. M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles. Cost function dependent barren plateaus in shallow parametrized quantum circuits. *Nature Communications*, 12(1):1791, 2021.
29. James C. Spall. An overview of the simultaneous perturbation method for efficient optimization. *Johns Hopkins APL Technical Digest*, 19(4):482–492, 1998.
30. Alex Galda, Prineha Narang, and Ilya Safro. Gradient-free optimization for variational quantum algorithms. *npj Quantum Information*, 9(1):45, 2023.
31. Andrea Mari, Thomas R. Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340, 2020.
32. Suguru Endo, Zi-Wen Cai, Simon C. Benjamin, and Xiao Yuan. Hybrid quantum-classical algorithms and error mitigation. *Journal of the Physical Society of Japan*, 90(3):032001, 2021.
33. Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Open quantum problems: Machine learning and variational quantum algorithms. *arXiv preprint arXiv:1803.11173*, 2019.
34. Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. The effect of data encoding on the expressive power of variational quantum machine learning models. *Physical Review A*, 103(3):032430, 2021.
35. Hsin-Yuan Huang, Richard Kueng, and John Preskill. Quantum advantage in learning from experiments. *Science*, 376(6598):1182–1186, 2022.
36. Andrew Arrasmith, M. Cerezo, Piotr Czarnik, Lukasz Cincio, and Patrick J. Coles. Equivalence of quantum barren plateaus to cost concentration and narrow gorges. *Quantum Science and Technology*, 6(6):064005, 2021.

37. Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. The quest for a quantum neural network. *Quantum Information Processing*, 13(11):2567–2586, 2014.
38. Sam McArdle, Suguru Endo, Alán Aspuru-Guzik, Simon C. Benjamin, and Xiao Yuan. Quantum computational chemistry. *Reviews of Modern Physics*, 92(1):015003, 2020.
39. Edward Grant, Marcello Benedetti, Shuxiang Cao, Andrew Hallam, Jack Lockhart, Vid Stojevic, Anthony G. Green, and Simone Severini. Hierarchical quantum classifiers. *npj Quantum Information*, 4(1):65, 2018.
40. Nikolaj Moll, Panagiotis Barkoutsos, Lev S. Bishop, Jerry M. Chow, Andrew Cross, Daniel J. Egger, Stefan Filipp, Andreas Fuhrer, Jay M. Gambetta, Marc Ganzhorn, et al. Quantum optimization using variational algorithms on near-term quantum devices. *Quantum Science and Technology*, 3(3):030503, 2018.
41. Yadong Wu, Juan Yao, Pengfei Zhang, and Hui Zhai. Expressivity of quantum neural networks. *Physical Review Research*, 3(3):L032049, 2021.
42. Jonathan Romero, Jonathan P. Olson, and Alán Aspuru-Guzik. Quantum generative adversarial networks. *Quantum Science and Technology*, 4(1):014008, 2019.
43. Patrick Rebentrost, Masoud Mohseni, and Seth Lloyd. Quantum support vector machine for big data classification. *Physical Review Letters*, 113(13):130503, 2014.
44. Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Carsten Blank, Keri McKiernan, and Nathan Killoran. Pennylane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018.
45. Qiskit Community. Qiskit machine learning. https://qiskit.org/documentation/machine-learning/, 2023. Accessed: 2025-09-16.
46. Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J. Martinez, Jae Hyeon Yoo, Sergei V. Isakov, Patrick Massey, Roozbeh Halavati, Murphy Yuezhen Niu, Alexander Zlokapa, et al. Tensorflow quantum: A software framework for quantum machine learning. *arXiv preprint arXiv:2003.02989*, 2020.